So, we need to think out of the box to handle this: since there is a strong random component that influence the result of each run, we can run canopy clustering several times, and take the result that is closer to our expectation. This might not be enough, though, because the variance between different runs is limited, and if we start with the “wrong” values for the thresholds, the algorithm could always output too many (or too few) centroids.

To solve this issue, we have two ways: either manually tuning these thresholds after each run, or, alternatively, perform some kind of search in the domain of the thresholds, trying different values for them, either by adding at each run a random value to our initial choice for T_1 and T_2, or tuning the thresholds depending on the number of canopies returned (lowering T_2 when too few centroids are selected in the last run, raising it when we get too many). If we’d like to get really fancy, we can even use ML to find the best values.

While the former idea is a brute-force fully-randomized search, a better targeted solution seems more promising, as we would direct the search towards values that should work better for our goal: we can use the same idea behind *gradient descent*[14], although with a simpler algorithm that just decides the direction of the update, without worrying about slopes and gradients. For instance, we could run a cycle where we adjust the value for the inner threshold (and, when needed, also the one for the outer threshold) depending on the difference between the result we get from canopy clustering and the number of centroids we need.